BỘ GIÁO DỤC VÀ ĐÀO TẠOTRƯỜNG ĐẠI
HỌC CÔNG NGHỆ KỸ THUẬT TPHCM

----ꕥ🕮ꕥ----

HCMUTE

COURSE: DATA STRUCTURES AND ALGORITHMS

REPORT

Minimum Spanning Tree with Union-Find (Kruskal's Algorithm)

Member	Student ID
Hà Minh Thuận	25110061
Lê Nguyễn Trọng Phúc	25110051

Ho Chi Minh City, October 2026

Team Members and Completion

Course: Data Structures and Algorithms

Report title: Minimum Spanning Tree with Union-Find (Kruskal's Algorithm)

Member	**Student ID**	**Completion**
Hà Minh Thuận	25110061	100%
Lê Nguyễn Trọng Phúc	25110051	100%

Table of Contents

1 Problem and research objectives....4

1.1 Context....4

1.2 Objectives....4

1.3 Methodology....4

1.4 Scope and data contract....5

2 Graph foundations and the greedy principle....6

2.1 Spanning trees and forests....6

2.2 The cut property....6

2.3 Negative weights and special edges....6

2.4 Union Find and its invariants....7

2.5 The two Union Find optimizations....8

Union by size....8

Path compression....8

2.6 Kruskal and its correctness....9

2.7 Complexity analysis....10

3 Program architecture....11

3.1 Representation and input validation....12

3.2 Generator and reproducibility....13

3.3 Win32 interface and workflow....14

3.4 Coordinates and visualization....15

3.5 Responsiveness and implementation limits....16

4 Testing strategy....17

4.1 Hand-traced example....18

4.2 Independent exhaustive oracle....19

4.3 Simulated 64-node network....20

4.4 The 20,000-vertex input....21

4.5 Design of the find benchmark....22

4.6 Benchmark results and interpretation....23

4.7 Component and whole-program performance 24
5 Self explanation of technical requirements 25
5.1 Disjoint-set structure 25
5.2 Kruskal with a custom structure 25
5.3 Compression comparison 25
5.4 A network of at least 50 nodes 25
5.5 Near-constant cost 25
5.6 Overall evaluation and conclusion 26
References and evidence 27

1 Problem and research objectives

1.1 Context

In a network model, vertices may represent routers, stations, or locations requiring connectivity, while edge weights represent link costs. If the only objective is connectivity at minimum total cost, a minimum spanning tree is an appropriate model. It does not automatically guarantee bandwidth, latency, or redundancy after a link failure. Those properties require additional constraints outside the scope of the MST solution.

Let G = (V, E), n = |V|, and m = |E|. Each edge e has endpoints u and v and an integer weight w(e). For a connected graph, the objective is to find T such that (V, T) is a tree and W(T) = Σ w(e), for e in T, is minimum among all spanning trees. The set T contains exactly n − 1 edges. This minimizes total network cost rather than the length of a shortest path between two vertices.

1.2 Objectives

The algorithmic objective is to implement Union-Find, use it in Kruskal's algorithm, and validate the solution through independent methods. The experimental objective is to observe the effect of compression over many find() queries, rather than infer it solely from a small example or GUI response time.

The software objective is to let users enter, generate, save, and inspect graphs; run both compression modes; and examine selected edges, total weight, and connectivity. The core is separated from the interface so that testing and benchmarking do not depend on mouse interaction.

1.3 Methodology

The report combines algorithm analysis with empirical verification. Union-Find invariants and the cut property establish the basis of correctness. Hand tracing, Prim's algorithm, and exhaustive search provide implementation checks. Large inputs and repeated measurements characterize execution under specified conditions.

The study evaluates partition correctness, MST optimality, and the effect of compression on both isolated find queries and complete solver execution. Correctness is examined using mathematical invariants and independent reference methods, while performance is evaluated using repeated measurements on controlled workloads.

1.4 Scope and data contract

The project considers undirected graphs with integer edge weights. Each edge is stored once as u, v, w. The core accepts parallel edges, self-loops, and negative weights. The random generator produces simple graphs without loops or repeated endpoint pairs. The algorithm's accepted input domain must therefore be distinguished from the type of input deliberately produced by the generator.

The standard format begins with n and m, followed by exactly m triples. Vertex indices range from 0 to n − 1. GraphInput checks counts, indices, missing data, extra data, and malformed tokens. The current limits are 1,000,000 vertices and 2,000,000 edges. These are acceptance limits, not guarantees of execution speed or interactive rendering at every size.

```
4 5
0 1 1
1 2 2
0 2 3
2 3 4
1 3 5
```

MSTResult contains the selected edges, total weight, component count, and a flag indicating whether a spanning tree exists. For a connected graph with $n \geq 1$, the result has one component and $n - 1$ edges. For a disconnected graph, the program returns a minimum spanning forest; with c components, it contains $n - c$ edges.

A single vertex without edges has an empty spanning tree of weight 0. The program treats a graph with no vertices as having no spanning tree, although the sum of an empty edge set remains 0. Stating this convention distinguishes the cases and prevents empty input from being interpreted as a connected network.

The GUI supports alphabetic and Unicode vertex names while retaining numeric IDs in the algorithmic core. Renaming affects presentation only and leaves topology, connectivity, and MST weight unchanged. GUI-saved files can include name annotations, whereas the console parser accepts the standard numeric format.

2 Graph foundations and the greedy principle

2.1 Spanning trees and forests

An undirected tree is connected and acyclic. A tree on n vertices contains n − 1 edges. Connectivity together with exactly n − 1 edges underlies the brute-force checker. A forest consists of disjoint trees; a forest with n vertices and c components contains n − c edges.

An MST need not be unique when edge weights are equal. Correctness is therefore assessed through total weight, connectivity, edge count, and acyclicity. Secondary ordering by endpoints provides deterministic tie breaking for a fixed input without changing the optimal total weight.

2.2 The cut property

A cut partitions the vertices into nonempty sets S and V\S. An edge crosses the cut when its endpoints belong to different sides. If the selected edge set respects the cut, a minimum-weight crossing edge is safe: adding it preserves the existence of an MST containing the current selected edges [1].

The exchange argument is as follows. Consider an MST containing the selected edges. If it already contains the candidate edge, no change is needed. Otherwise, adding that edge creates a cycle containing another crossing edge. Replacing the latter with the minimum-weight edge does not increase total weight and restores a tree. Thus, the greedy choice can still be extended to an optimum.

2.3 Negative weights and special edges

Kruskal requires an ordering of weights and addition of totals, so negative weights do not invalidate the cut property. A self-loop is rejected because its endpoints are always in the same component. Parallel edges remain separate choices; once the relevant components have been connected, another such edge can be rejected because it creates a cycle.

MST optimization minimizes the sum of selected link weights. It does not minimize the distance between every pair of vertices; consequently, an MST and a shortest-path tree generally solve different optimization problems.

2.4 Union Find and its invariants

Union-Find maintains a partition of the vertices: every element belongs to exactly one set, the sets are disjoint, and together they cover all elements. findRoot(x) returns the representative of the set containing x; unite(x, y) merges two sets if they differ. In Kruskal, each set represents a component of the currently selected forest.

The implementation uses parent to store parent links and setSize to store sizes at roots. A vertex r is a root when parent[r] = r. Initially, parent[i] = i, setSize[i] = 1, and componentCount = n. The size stored at a vertex that is no longer a root is not used to decide subsequent merges.



Figure 1 A parent-chain branch before and after path compression

The principal invariants are that every parent chain terminates at a valid root, two vertices belong to the same set exactly when they have the same root, and each successful merge decreases the component count by one. A false return from unite leaves the partition unchanged, although find operations may have compressed some parent links.

Compression changes representation without transferring a vertex to another set. Merging different roots changes the partition. The parent array is not the set of MST edges: it describes the supporting data structure. MST edges are stored separately in the result.

Indices are checked before array access. unite validates both arguments before searching for roots, avoiding partial compression caused by a valid first argument followed by an invalid second argument. An exception lets the caller stop and report invalid data instead of continuing with an unreliable computation.

2.5 The two Union Find optimizations

Union by size

After obtaining roots r_1 and r_2, unite returns without merging when they coincide. Otherwise, the smaller tree is attached to the larger root, set sizes are combined, and the component count decreases. Union-by-size uses explicit component cardinalities to control tree growth.

Whenever an element becomes deeper because its tree is attached below another, its new set is at least twice as large. A set cannot exceed n elements, so depth increases at most approximately $\log_2 n$ times. Union-by-size alone therefore bounds tree height by O(log n), instead of the O(n) chain possible under arbitrary merging.

Path compression

findRoot uses two iterative passes. The first follows parent links to locate the root. If compression is enabled, the second revisits the branch and makes its vertices point directly to the root. Iteration avoids a recursive call stack proportional to tree depth.

```
findRoot(x):
  r ← x
  while parent[r] ≠ r: r ← parent[r]
  if compression:
    while parent[x] ≠ x:
      next ← parent[x]
      parent[x] ← r
      x ← next
  return r
```

Disabling compression skips parent updates; union-by-size remains active. The benchmark therefore compares variants sharing a merge rule and query sequence but differing in compression. This isolates its effect better than simultaneously changing both merging and root finding.

Compression preserves set membership and the MST solution. Every updated vertex still reaches the same representative. Early queries may traverse several links, while later queries benefit from a flatter structure. The additional update cost must also be considered when interpreting actual execution times.

2.6 Kruskal and its correctness

Kruskal starts with an empty edge set and n singleton components. It sorts edges by increasing weight and examines each edge u–v. If the endpoints belong to different sets, the edge is added and the sets are merged. Otherwise, the current forest already contains a path between the endpoints; adding the edge would create a cycle, so it is rejected.

```
Kruskal(G):
   validate G; sort E by weight
   initialize UnionFind(n)
   T ← empty; W ← 0
   for each sorted edge (u, v, w):
      if unite(u, v):
         add edge to T; W ← W + w
         if |T| = n − 1: stop
   return T, W, and the component count
```

Acyclicity is preserved because edges are added only between different components. Consider an accepted edge and the component containing u immediately before the merge. The edge is a minimum-weight crossing edge under the processing order. A lighter edge previously rejected because its endpoints were in the same set cannot cross the current component. By the cut property, the accepted edge is safe. Induction shows that the selected set can always be extended to an MST.

If the graph is connected, the process reaches n − 1 edges and one component, producing an optimal spanning tree. Otherwise, no remaining edge connects the final components, and the same reasoning within each component yields a minimum spanning forest. hasSpanningTree distinguishes these outcomes instead of presenting a forest as a spanning tree of the whole graph.

The implementation separates sortEdges and solveSorted for stage-level measurement. The normal entry point, solve, validates, copies, sorts, and processes edges. solveSorted checks ordering before execution and rejects an unsorted list. This is essential because an incorrect order may still produce a tree without guaranteeing optimality.

Secondary ordering by from and to breaks equal-weight ties. Correctness does not require distinct weights; any order among tied edges is valid as long as nondecreasing weight order is preserved.

2.7 Complexity analysis

Union-Find initialization requires O(n) time and memory. With union-by-size alone, findRoot has O(log n) worst-case cost. Combined with compression, the amortized cost per operation is O(α(n)), where α is a form of the inverse Ackermann function. The near-constant behavior of the combined structure is described in the Union-Find reference [2].

Amortized analysis bounds the total cost of an operation sequence rather than requiring constant cost for each individual query. An initial find may traverse several parent links, while subsequent queries benefit from compression. The established O(α(n)) bound characterizes the combined effect of size-based merging and path compression; the empirical measurements assess its practical consequences.

Component	Time	Principal memory
Set initialization	O(n)	O(n)
Find with size only	O(log n) worst case	O(1) auxiliary
Operation with compression	O(α(n)) amortized	O(1) beyond arrays
Sorting m edges	O(m log m)	Sorting dependent
Kruskal and graph data	O(n + m log m + m α(n))	O(n + m)

Validation and copying are linear; sorting requires O(m log m); set processing requires O(m α(n)). The total is often simplified to O(m log m) when sorting dominates. The O(n) term must remain when discussing graphs with many isolated vertices or very few edges. Without compression, set processing can be bounded by O(m log n).

The total weight uses long long to avoid overflow when adding many int weights. With at most one million vertices, the forest contains fewer than one million edges; the sum of their int weights fits a signed 64-bit integer. The GUI additionally stores coordinates, labels, and selection state. Its rendering costs are separate from the complexity of the MST algorithm.

3 Program architecture

The dsa_core library contains graph representation, input validation, Union-Find, Kruskal, and the generator. The console, GUI, benchmarks, and tests share this library. This reduces the risk of separate algorithm implementations and permits core evaluation without opening an interactive window.

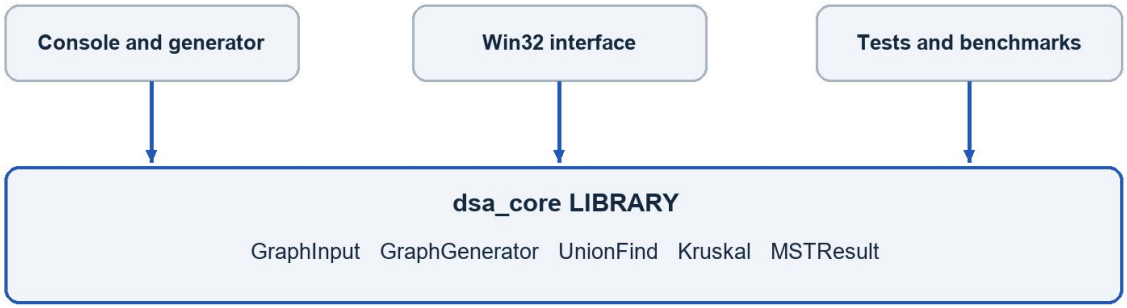


Figure 2 Programs sharing the algorithm library

Files	Responsibility
Edge.h and Graph.h	Edges, vertex count, and edge list
MSTResult.h	Solution, weight, and components
UnionFind.h and UnionFind.cpp	Partition and root-finding modes
Kruskal.h and Kruskal.cpp	Sorting and optimal edge selection
GraphInput.h and GraphInput.cpp	Reading and validation
GraphGenerator.h and GraphGenerator.cpp	Parameterized data generation
GraphView and VertexLabels	Rendering, interaction, and names
Benchmarks and tests	Timing and independent checks

Core types do not contain HWND or HDC. Changing the view updates only GraphViewState. Changing topology or weights must invalidate the previous solution. This boundary separates presentation errors from algorithm errors and explains why zooming must not rerun Kruskal.

CMake defines C++11 build targets for the core library, console application, GUI, graph generator, benchmarks, and test executables. The GUI build copies sample data into the executable directory to support direct loading of predefined inputs.

3.1 Representation and input validation

Edge contains int fields from, to, and weight. Graph contains vertexCount and an edge vector; screen coordinates are not vertex IDs. An edge list suits Kruskal because sorting and traversal are its main operations, rather than repeated neighborhood queries. Compared with an $O(n^2)$ adjacency matrix, this is more appropriate for sparse networks and the 20,000-vertex input.

MSTResult separates the solution from the input. solve copies the graph before sorting, allowing the edge table to preserve entry order. Coloring the MST requires mapping solution edges to original edges, particularly for equal-weight parallel edges. A position in the sorted list cannot automatically be treated as the original table position.

GraphInput::read requires exactly the declared data. Missing weights, nonnumeric tokens, negative indices, and an index equal to n are rejected. Extra data after m edges are invalid as well. Shared validation makes GUI, console, and test callers follow the same contract.

Situation	Handling
Negative n or m	Reject
Endpoint outside the domain	Reject before array access
Negative weight	Accept
Self-loop	Valid, but not selected for the MST
Equal weights	Multiple optima may exist
Disconnected graph	Return a forest and component count

Input validation distinguishes structural properties from malformed data. A disconnected graph is accepted and produces a minimum spanning forest, whereas an invalid vertex index or incomplete edge record is rejected before solving.

Strict parsing ensures that a file either defines the complete declared graph or is rejected. Requiring exactly m edge records prevents incomplete or surplus input from silently changing the graph used in an experiment.

3.2 Generator and reproducibility

The generator accepts n, m, minimum and maximum weights, a seed, and a connectivity option. For a simple undirected graph, m cannot exceed $n(n-1)/2$. Guaranteed connectivity requires positive n and $m \geq n-1$. The lower weight bound must not exceed the upper bound. These conditions are checked before generation.

To guarantee connectivity, the program shuffles vertices and connects each new vertex to an earlier one. The initial edges form a spanning tree. It then adds unused pairs until m edges are present. Endpoint pairs are normalized with the smaller endpoint first, and a set prevents duplicate undirected edges.

For sparse graphs, random endpoints are sampled and unused pairs are added. Dense graphs can encounter many duplicate attempts, so the code enumerates missing pairs, shuffles them, and takes enough. This optimizes preparation rather than changing Kruskal's complexity.

DSA_generate V E minWeight maxWeight seed connected output.txt
DSA_generate 20000 100000 -1000 1000 42 1 large_20000_100000.txt

A connectivity flag of 1 constructs a connected graph; a flag of 0 permits either outcome. The fixed seed controls generation within a given toolchain. The exact generated file provides the experimental input because standard-library implementations can differ in distribution and shuffle details.

The structured 64-node network represents a simulated application, while the 20,000-vertex graph evaluates scalability. The generator first constructs a spanning tree when connectivity is enabled, so its output distribution is not uniform over all connected graphs with the same n and m.

The generation parameters and command identify the workload construction procedure. Retaining the generated file additionally fixes the exact edge set and weights used in the reported measurements.

3.3 Win32 interface and workflow

The graphical interface uses the Win32 API and GDI/GDI+ rendering. Native controls, window messages, and file dialogs provide the interaction layer, while the C++11 core remains independent of a higher-level GUI framework.

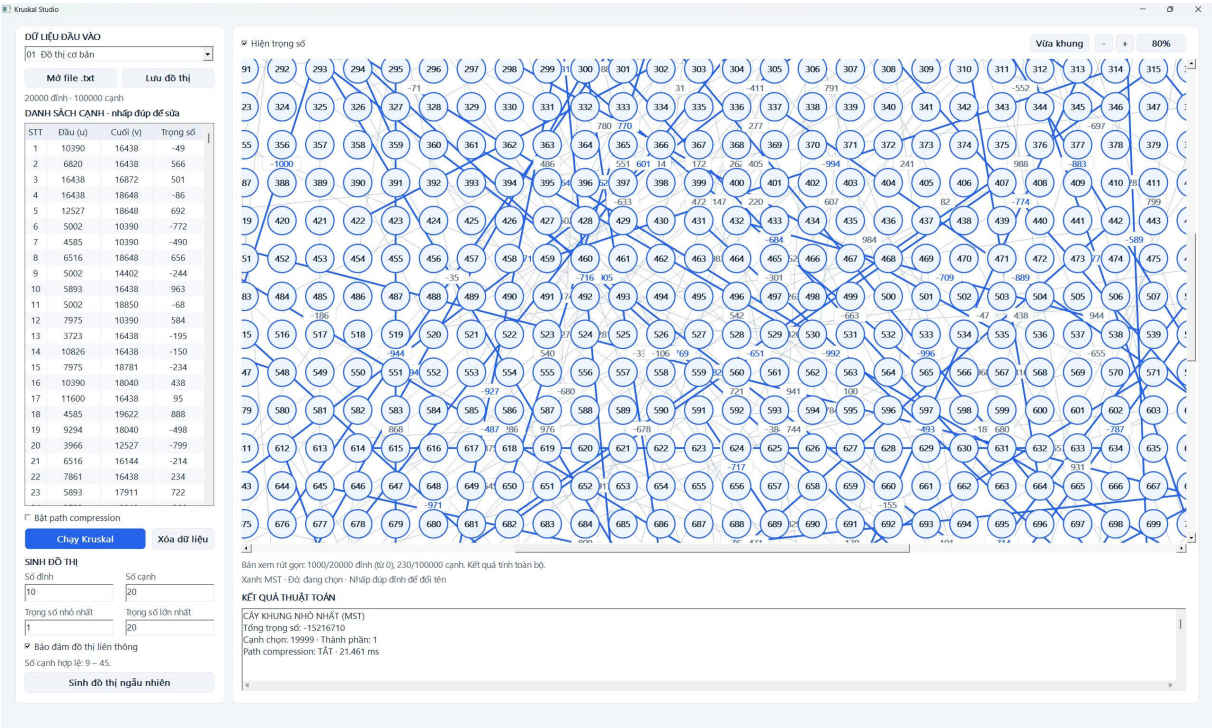


Figure 3 GUI and large-graph result with compression disabled

The left panel concentrates data operations: samples, opening and saving files, the edge table, compression, execution, and generation settings. The right side contains the canvas, weight visibility, fit-to-view, zoom, and scrollbars. Results appear below the canvas to connect visual inspection with quantitative output.

The normal workflow is to load data, inspect the edge table, select a mode, and run Kruskal. Selected solution edges are highlighted; the result reports weight, edge count, components, and elapsed time. Changing zoom or the visible region does not require solving again.

For large inputs, the interface reports the size of the reduced preview separately from the complete graph. Rendering uses the displayed subset, whereas MST weight, selected-edge count, and connectivity are computed from the full input.

The interface supports vertex renaming for presentation. Display names and algorithm IDs remain separate so that renaming does not change the mathematical problem. This also allows editing tests to distinguish Unicode handling from Union-Find correctness.

3.4 Coordinates and visualization

GraphViewState stores logical coordinates, zoom, translation, selection, and pointer state. Coordinates are mapped consistently onto the canvas, and mouse selection must use corresponding geometry. Inconsistent rendering and hit-testing coordinates cause misplaced selections or labels at different DPI settings.

The implementation uses logical units based on 96 DPI. The DPI/96 factor relates logical units to physical pixels. Microsoft explains this distinction and why bitmap scaling after drawing may blur text [3]. Fonts, vertex radii, layout, and hit testing must scale together, rather than changing only window dimensions.

The default layout is a grid. Coordinates aid inspection and do not represent geography or weight. A long screen edge need not have a high cost. Vertices remain readable; nearby screen points must not change the number of actual graph vertices.

Parallel edges use separate paths and self-loops use curves. Edge labels have independent geometry for rendering and hit testing. Selection highlights the chosen edge while preserving the remaining edges, and fit-to-view accounts for curved paths and label bounds.

The reduced preview is limited to 1,000 vertices and 3,000 edges within the displayed domain. This controls rendering cost and overlap; the core still solves the complete input. Fit-to-view centers the rendered subset, not necessarily the entire large network.

Renaming does not change IDs or the solution. Deleting a vertex must remove incident edges and consistently remap remaining IDs. Changing edges or weights invalidates the previous solution; renaming and viewing changes do not require recomputation. These rules maintain consistency among the table, canvas, and result.

3.5 Responsiveness and implementation limits

Reading, generating, saving, and solving large graphs may take noticeable time. The GUI delegates heavy tasks to a worker thread and transfers completion results to the interface thread through messages. Incompatible editing operations are disabled during processing. Closing the window must safely finish the worker lifecycle before related resources are destroyed.

This does not make Kruskal a parallel algorithm. The benefit is preserving message-loop responsiveness. Disabling edits also prevents a result from being computed from data modified during execution. Responsiveness and algorithm execution time are distinct evaluation concerns.

The GUI limits parallel edges per pair to preserve path distinguishability. The algorithmic core accepts a broader input domain. For very large solutions, the displayed list is truncated while weight, edge count, and component count retain their complete values.

A grid does not minimize crossings for every graph. Dense graphs remain difficult to read with all weights enabled. Zooming, hiding labels, and inspecting the table help, but do not remove finite screen-space constraints. The software prioritizes correctness and an explicit reduced preview rather than shrinking vertices until labels become unreadable.

Rendering tests address label visibility, curved-edge bounds, DPI-dependent geometry, and selection behavior. Integration tests cover graph editing and result consistency. These checks complement algorithm tests by evaluating how computed results are presented and updated during interaction.

The separation of Union-Find, Kruskal, input validation, generation, and rendering allows each component to be tested independently. This organization also localizes changes: modifying the view does not require changes to the optimization algorithm.

4 Testing strategy

Testing proceeds from units to integration. The first layer covers set operations and simple solutions. The next covers input contracts, edge cases, and Prim comparisons. The final layer covers generation, exhaustive checking, and the GUI. This structure narrows the cause when a case fails.

CTest suite	Scope
Week1Core	Basic Union-Find and Kruskal
Week2Validation	Invalid data, edge cases, and Prim
Week3Generator	Generator, seed, and sorting API
BruteForceMST	Three fixed cases and 100 small graphs
Week3DpiRendering	Geometry, DPI, labels, and selection
GuiIntegration	Interaction, data, and GUI integration

All six automated CTest suites passed in the verification run conducted on November 2, 2026. The suites cover core operations, validation, generation, exhaustive comparison, rendering, and GUI integration. The result establishes successful execution of the defined tests, subject to their input and interaction coverage.

Solution validation checks independently accumulated weight, selected-edge count, connectivity, and acyclicity. Connectivity together with $n - 1$ edges establishes a tree on a connected input; comparison with an independent oracle establishes optimality for the examined instances.

The test cases include empty graphs, singleton and isolated vertices, disconnected inputs, parallel edges, self-loops, equal and negative weights, and totals exceeding the 32-bit range. Malformed-input cases verify rejection before solving, with explicit expected outcomes.

Randomized cases extend the set of examined graph structures. Their finite coverage complements the correctness argument and the fixed exhaustive cases rather than constituting a proof over all possible inputs.

4.1 Hand-traced example

Consider four vertices and five edges: 0–1 with weight 1; 1–2 with weight 2; 0–2 with weight 3; 2–3 with weight 4; and 1–3 with weight 5. This input is small enough to verify each decision without relying on program output.

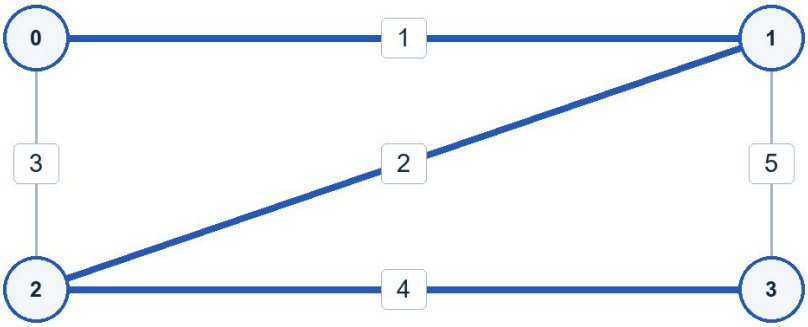


Figure 4 The three MST edges in the hand-traced example

Step	Edge	Decision	Reason
1	0–1, w = 1	Accept	Joins singleton sets
2	1–2, w = 2	Accept	Vertex 2 is not connected yet
3	0–2, w = 3	Reject	Path 0–1–2 already exists
4	2–3, w = 4	Accept	Connects remaining vertex 3
5	1–3, w = 5	Stop beforehand	Already n − 1 edges

The result is T = {0–1, 1–2, 2–3} with W(T) = 1 + 2 + 4 = 7. Three successful merges reduce the number of sets from four to one. The result is acyclic and connects every vertex. Stopping at n − 1 edges is justified because every accepted edge previously joined distinct components.

Kruskal does not simply take the n − 1 lightest edges. Weights 1, 2, and 3 form a cycle among vertices 0, 1, and 2 while excluding vertex 3. Union-Find rejects weight 3 and allows weight 4, making the result a tree on the complete vertex set.

The hand trace is recorded with small_graph.txt in expected_results.txt. Both interface and console execution use the same core solver. Vertex renaming and layout changes preserve the graph structure and therefore preserve the MST weight of 7.

4.2 Independent exhaustive oracle

Brute-force enumerates every subset of exactly n − 1 edges from m edges. For each subset, the program creates its own adjacency list and uses DFS to count reachable vertices. Reaching all n establishes connectivity; together with n − 1 edges, this establishes a spanning tree. The minimum sum across valid trees is retained.

The oracle does not use Union-Find to determine connectivity. Reusing the structure under validation could allow a shared defect to produce matching but incorrect results. DFS supplies a structurally different check, strengthening independence from Kruskal.

n	m	Candidate subsets	Valid trees	Optimal weight
6	8	C(8,5) = 56	30	11
7	10	C(10,6) = 210	79	12
8	12	C(12,7) = 792	207	14

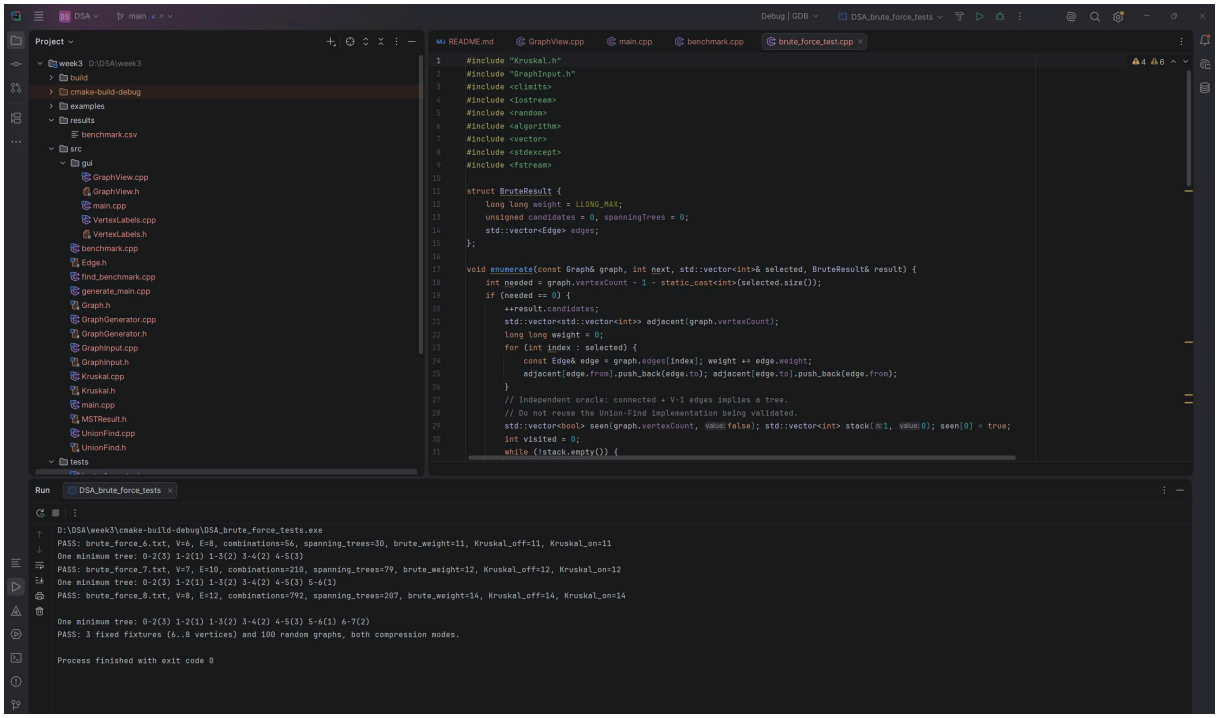


Figure 5 Exhaustive results and Kruskal comparisons

All three totals match Kruskal with compression OFF and ON. An additional 100 small random graphs are checked in both modes. Candidate count differs from tree count because disconnected subsets are rejected. Keeping the quantities separate accurately describes workload and valid solutions.

The exhaustive checker examines C(m, n − 1) candidate subsets, each requiring adjacency construction and traversal. Its implementation is limited to 8 vertices and 16

edges. This makes it suitable as an independent correctness oracle for small graphs, while Kruskal handles the larger workloads.

4.3 Simulated 64-node network

network_64.txt contains 64 vertices and 161 links based on an 8 × 8 grid and neighboring connections. Weights represent simulated costs. The input illustrates a network application but does not represent every distribution of weights or real-world link quality.

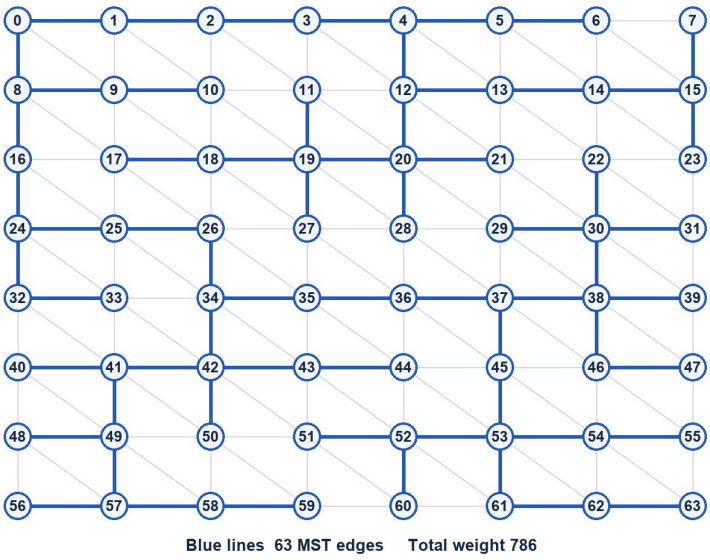


Figure 6 Network structure drawn from the project's 64-node edge file

expected_results.txt records 63 selected edges, total weight 786, and one component. The count agrees with n − 1. The complete list of 63 edges allows readers to recompute the total and check connectivity rather than trust only the final printed number.

Quantity	Value
Input nodes	64
Input links	161
MST edges	63
Total weight	786
Components	1

The solution connects every node at minimum total simulated cost. A tree has no redundant link: removing any edge divides it into two components. A total of 786 therefore does not establish an optimal design for a fault-tolerant network or one requiring alternative routes.

The 64-node case provides a structured network application beyond the hand-traced examples. Grid coordinates are illustrative; optimization uses the weights stored in the edge file rather than geometric edge lengths.

4.4 The 20,000-vertex input

The large input contains 20,000 vertices and 100,000 edges, with weights from −1,000 to 1,000, seed 42, and guaranteed connectivity. It evaluates input handling and solver execution at a scale substantially larger than the hand-traced example. The generation procedure and command are described in Section 3.2.

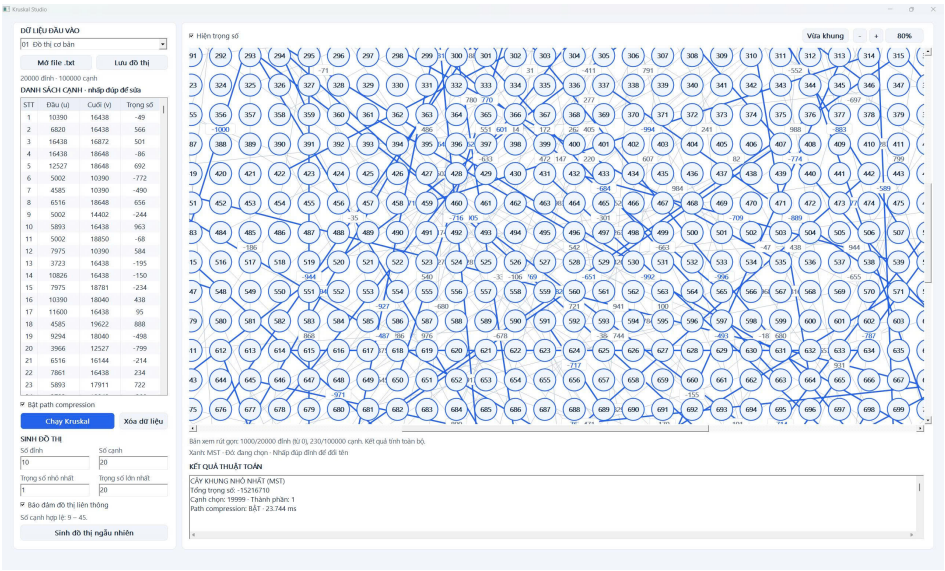


Figure 7 The same large input with compression enabled

Quantity	OFF	ON
Vertices	20,000	20,000
Input edges	100,000	100,000
Result edges	19,999	19,999
Total weight	−15,216,710	−15,216,710
Components	1	1
Single GUI run	21.461 ms	23.744 ms

Both modes return the same solution quantities. Count and connectivity agree with a tree, and a negative total is consistent with negative input weights. Agreement between modes is a consistency check, not exhaustive proof of optimality on the large input. Optimality rests on Kruskal's correctness argument and independent checks on smaller inputs.

The screenshot explicitly identifies a preview of 1,000/20,000 vertices and 230/100,000 edges. Preview edge count depends on endpoints within the displayed

vertex subset; it does not mean the algorithm processes only 230 edges. The 19,999 result edges are computed from the full input.

The GUI measurements are individual observations rather than medians across repeated trials. Compression-enabled execution took 23.744 ms, compared with 21.461 ms when disabled. These observations motivate the separate component-level analysis presented below.

4.5 Design of the find benchmark

find_benchmark.cpp uses 65,536 elements and 1,000,000 queries per trial. Seed 42 generates a shared query sequence. Before each measurement, the program reconstructs the same structure through balanced merges of groups of size 1, 2, 4, and successive powers of two.

This creates multiple depth levels despite union-by-size. If every element already pointed directly to its root, compression would have little work left to eliminate. The initial structure is therefore part of the benchmark design that must be explained when evaluating optimization benefits.

Query generation, allocation, and set construction are outside the timed find region. steady_clock measures the complete query batch, then divides by one million. The ns/find unit is a batch average, not a claim that every individual call is timed at exactly one-nanosecond resolution.

t_j = query-batch duration in trial j / 1,000,000
t_reported = median(t_1, t_2, ..., t_9)
speedup = t_OFF / t_ON

One warm-up trial is excluded, followed by nine measured trials. OFF/ON order alternates to reduce bias from always running one mode first. Every trial starts with a fresh structure, preventing compression from an earlier trial from changing the next trial's starting state.

A checksum accumulates returned roots and is compared across modes. It checks query consistency and ensures computed results are used in the output. It does not replace data-structure tests but is a useful experimental control.

For each mode, the nine per-trial average times are sorted and the fifth value is reported as the median. This statistic summarizes typical batch performance while reducing sensitivity to isolated timing outliers.

4.6 Benchmark results and interpretation

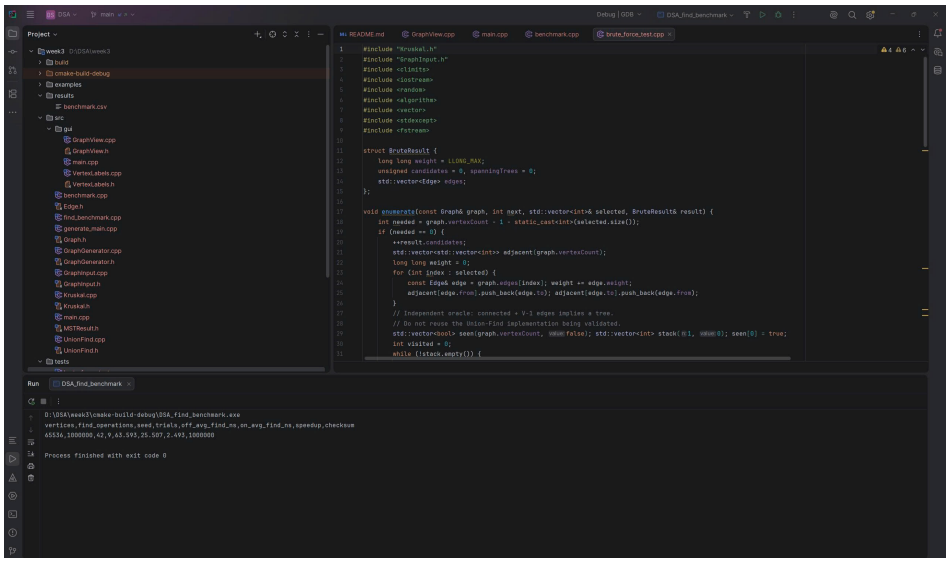


Figure 8 The find benchmark under the CLion Debug configuration

Parameter	Recorded value
Elements	65,536
Find operations per trial	1,000,000
Measured trials after warm-up	9
Seed	42
Median per-trial average find time OFF	63.593 ns
Median per-trial average find time ON	25.507 ns
OFF divided by ON	Approximately 2.493
Checksum	1,000,000



Figure 9 Comparison based on the recorded Debug measurements

Under the selected workload, compression reduces find time according to the defined metric. Early queries flatten branches, allowing later ones to traverse fewer links. The measurement supports this observation, but not a universal speedup of 2.493 for every input.

The reported measurements were obtained in Debug mode. Execution time depends on compiler settings, hardware, system load, and the initial parent-tree structure; the observed speedup is therefore specific to the measured workload and environment.

4.7 Component and whole-program performance

The micro-benchmark isolates find queries on a prepared structure, whereas solve includes validation, copying, sorting, initialization, edge processing, and result construction. Improving find need not dominate total time. The two measurement levels answer different questions.

The large GUI run records 23.744 ms for ON and 21.461 ms for OFF. Union-by-size already keeps trees shallow; compression introduces parent writes. When paths are not reused enough or sorting dominates, find savings can be smaller than additional costs and measurement noise. These are plausible explanations, not effects independently isolated through hardware counters in this project.

benchmark.cpp measures sorting, processing presorted edges, and the complete solve. Vertex/edge sizes are 1,000/5,000, 5,000/25,000, 10,000/50,000, 20,000/100,000, and 20,000/300,000. Each case uses warm-up and nine trials, exports CSV medians, and verifies consistent weights, edge counts, and components.

solveSorted time is not pure find time: it includes validation, order checking, and result accumulation. Subtracting two independently computed medians does not give an exact component duration. The median of a sum need not equal the sum of medians; accurate trial-level decomposition requires measurements from the same trial.

The supported conclusion is that compression improves find on the published isolated workload but does not guarantee an overall Kruskal speedup in the GUI observations. A pair of timings neither proves nor disproves asymptotic complexity. Complexity concerns growth with input size and operation sequences; measurements also include constants and machine conditions.

The performance evaluation is limited to the recorded configuration and workloads. It does not establish a hardware-independent speedup or quantify the separate effects of cache behavior, parent writes, and scheduling noise.

5 Self explanation of technical requirements

5.1 Disjoint-set structure

The implementation combines parent trees, union-by-size, and iterative two-pass compression. Size-based merging bounds tree growth, while compression shortens traversed paths for later queries. Core tests verify the associated partition and component-count invariants.

5.2 Kruskal with a custom structure

The algorithm invokes unite on sorted edges. False indicates endpoints in the same component; true adds one edge, accumulates weight, and decreases the component count. Optimality follows from the cut property. Union-Find detects cycles rather than deciding which edge is lightest. Hand tracing and the exhaustive oracle check this decision sequence.

5.3 Compression comparison

Both modes share the merge rule, initial structure, and query sequence. A million find operations per trial provide a batch average, and the median of nine trials reduces sensitivity to unusual observations. Published Debug results yield approximately 2.493 times speedup for this workload, not a promise for every solve call.

5.4 A network of at least 50 nodes

The simulated network contains 64 nodes and 161 links. Kruskal selects 63 edges with total weight 786, with the complete edge list recorded in expected_results.txt. The larger 20,000-vertex workload separately evaluates input scale and consistency between compression modes.

5.5 Near-constant cost

Size doubling bounds depth increases during merging, and compression flattens traversed paths. Their combined amortized operation cost is $O(\alpha(n))$. Kruskal additionally sorts edges in $O(m \log m)$, which commonly dominates total execution time.

5.6 Overall evaluation and conclusion

The evaluation combines theoretical analysis with implementation tests. The cut property establishes the safety of Kruskal's choices, while partition invariants justify cycle detection. Hand tracing, Prim comparisons, and independent exhaustive enumeration validate the implementation on small inputs; larger workloads evaluate scalability and consistency.

Criterion	Evidence	Limitation
Hand tracing	Total 7 and edge decisions	Not all graphs
Exhaustive check	11/12/14, independent DFS	Small graphs only
Network example	64 nodes, total 786	Simulated data
Large-scale input	20,000 vertices, 100,000 edges	Reduced canvas preview
Compression	Debug speedup about 2.493	Workload dependent
Integration	Six CTest suites pass	Not a guarantee of no defects

Current limitations include a Windows-only GUI, grid crossings, reduced previews for large data, and experiments that are not a multi-machine study. Vertex names belong to the presentation layer, and input layers have their respective contracts. These limitations define the intended scope of use.

The completed application integrates graph input and editing, file loading and saving, vertex renaming, random graph generation, and interactive visualization with the Kruskal solver. Correctness was evaluated through hand tracing, comparison with Prim's algorithm, and independent exhaustive enumeration on small graphs. Testing also covered a simulated 64-node network and a larger graph containing 20,000 vertices and 100,000 edges. All six automated CTest suites passed. These results support the implementation's correctness within the tested cases and demonstrate the integration of the algorithmic core with the graphical interface, subject to the limitations stated above.

The project implements a minimum spanning tree solver whose algorithmic behavior is supported by correctness arguments and independent tests. Path compression improves the isolated find workload in the recorded experiment, while total Kruskal performance also depends on sorting and execution conditions. The resulting software combines a reusable algorithmic core with an interactive graph interface and reproducible test inputs.

References and evidence

The following references provide the theoretical and implementation background. The project repository provides access to the source code and experimental materials.

[1] Sedgewick, R., and Wayne, K. Algorithms Fourth Edition, Minimum Spanning Trees. Princeton University. https://algs4.cs.princeton.edu/43mst/index.php

[2] Sedgewick, R., and Wayne, K. UF class documentation. Algorithms Fourth Edition. https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/UF.html

[3] Microsoft Learn. DPI and device-independent pixels. https://learn.microsoft.com/en-us/windows/win32/learnwin32/dpi-and-device-independent-pixels

[4] Project repository. https://github.com/phuck3a222-jpg/DSA-Kruskal-UnionFind--25110051---25110061